

MULTITHREADING IN JAVA

1. Title

Multithreading – Implementing a Seat Reservation System Using Multiple Threads and Synchronization.

2. Aim

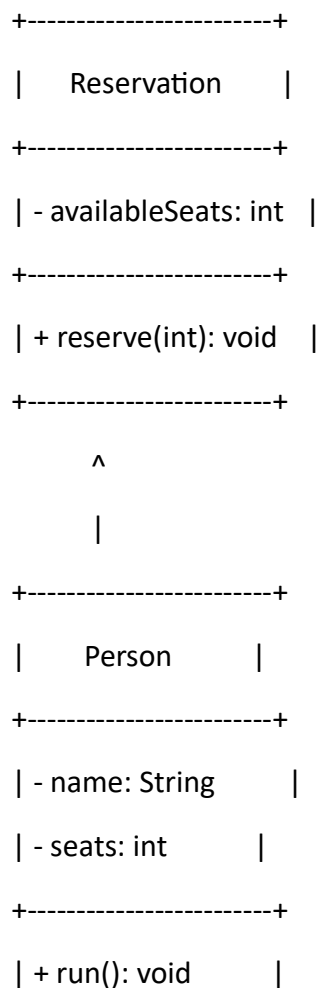
To implement a reservation system in Java using multiple person threads and a synchronized "reserve()" method to ensure that only one thread can access the method at a time.

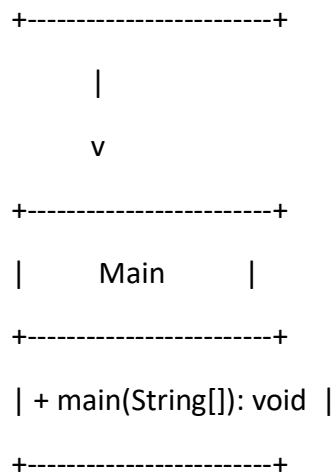
3. Software and Hardware Requirements

Software: Java JDK and Visual Studio Code / Eclipse IDE.

Hardware: Computer or laptop with a minimum of 4 GB RAM.

4. UML Diagram





5. Algorithm

1. Start the program.
2. Create a "Reservation" class with an initial availability of 10 seats.
3. Define a synchronized "reserve()" method to book seats.
4. Create a "Person" class that extends the "Thread" class.
5. Pass the person's name, requested seats, and reservation object to each thread.
6. Create three person threads requesting 5, 2, and 4 seats, respectively.
7. Start all threads.
8. The synchronized method allows only one thread to book seats at a time.
9. Display the reservation details and remaining seats.
10. Stop the program.

6. Source Code

```

class Reservation {
    int availableSeats = 10;

    synchronized void reserve(String name, int seats) {
        System.out.println(name + " entered.");
        System.out.println("Available seats: " + availableSeats
            + " Requested seats: " + seats);
        if (seats <= availableSeats) {

```

```

System.out.println("Seat Available. Reserve now :-)");
    availableSeats = availableSeats - seats;
    System.out.println(seats + " seats reserved.");
} else {
    System.out.println("Requested seats not available :-)");
}
System.out.println(name + " leaving.");
System.out.println("-----");
}
}

class Person extends Thread {
    Reservation reservation;
    int seats;

    Person(String name, Reservation reservation, int seats) {
        super(name);
        this.reservation = reservation;
        this.seats = seats;
    }

    public void run() {
        reservation.reserve(getName(), seats);
    }
}

public class ReservationDemo {
    public static void main(String[] args) {
        Reservation reservation = new Reservation();
        Person p1 = new Person("Person-1", reservation, 5);
        Person p2 = new Person("Person-2", reservation, 2);
        Person p3 = new Person("Person-3", reservation, 4);
    }
}

```

```
p1.start();
try {
    p1.join();
    p2.start();
    p2.join();
    p3.start();
    p3.join();
} catch (InterruptedException e) {
    System.out.println("Thread interrupted.");
}
}
```

7. Compilation and Execution Commands

```
javac ReservationDemo.java
```

```
java ReservationDemo
```

8. Output

Person-1 entered.

Available seats: 10 Requested seats: 5

Seat Available. Reserve now :-)

5 seats reserved.

Person-1 leaving.

Person-2 entered.

Available seats: 5 Requested seats: 2

Seat Available. Reserve now :-)

2 seats reserved.

Person-2 leaving.

Person-3 entered.

Available seats: 3 Requested seats: 4

Requested seats not available :-)

Person-3 leaving.

Output screenshot

```
-----
PS C:\Users\anits\Desktop\Java-Projects> javac ReservationDemo.java
PS C:\Users\anits\Desktop\Java-Projects> java ReservationDemo
Person-1 entered.
Available seats: 10 Requested seats: 5
Seat Available. Reserve now :-)
5 seats reserved.
Person-1 leaving.
-----
Person-2 entered.
Available seats: 5 Requested seats: 2
Seat Available. Reserve now :-)
2 seats reserved.
Person-2 leaving.
-----
Person-3 entered.
Available seats: 3 Requested seats: 4
Requested seats not available :-)
Person-3 leaving.
-----
PS C:\Users\anits\Desktop\Java-Projects> |
```

Result

Thus, the Java multithreading-based seat reservation system was implemented successfully using multiple threads and a synchronized reserve() method. The synchronized method ensures that only one thread can reserve seats at a time, preventing overbooking. Person-1 reserves 5 seats, Person-2 reserves 2 seats, and Person-3's request for 4 seats is rejected because only 3 seats remain.